

YallaMarket

Final Report — Comprehensive System Documentation

Software Engineering Course Project

University	An-Najah National University
Course	Software Engineering
Instructor	Dr. Firas Shakaa
Program / Major	Computer Science Apprenticeship
Semester	Second Semester
Methodology	Agile / Scrum
Technology Stack	React + Vite + Tailwind CSS, Express.js, SQLite, JWT
GitHub Repository	https://github.com/BaraaMasri99/yallaMarket

Team Members

Baraa Masri	Scrum Master / Development / Documentation
Hamed Bizreh	Development / Design / QA Evidence
Yahya Ziyoud	Development / Design / QA Evidence

Final Submission Version | May 2026

Contents

1. Executive Summary	4
2. Project Overview and Problem Definition.....	4
2.1 Problem Definition	4
2.2 Product Scope.....	4
3. Stakeholder Analysis.....	4
4. Requirements Summary.....	4
4.1 Functional Requirements Summary	4
4.2 Non-Functional Requirements Summary.....	5
5. System Architecture and Technology Stack.....	5
6. System Design Document (SDD)	6
6.1 Use Case Diagram.....	7
6.2 Class Diagram	8
6.3 Sequence Diagrams.....	9
6.3.1 Login / Logout Flow.....	Error! Bookmark not defined.
6.3.2 Language Switch Flow	10
7. Database Design	11
7.1 Entity Relationship Diagram	12
7.2 Data Model Summary	13
8. Implementation Overview.....	13
8.1 Frontend Implementation	13
8.2 Backend Implementation	13
8.3 API Summary	13
9. Quality Assurance.....	14
9.1 Test Plan Summary	14
9.2 Test Execution Summary.....	14
9.3 Bug Summary.....	14
9.4 Requirements Traceability Matrix.....	14
10. Agile Project Management Evidence	14
11. Version Control and Collaboration Evidence.....	15
12. Known Limitations and Future Enhancements	15
13. Final Submission Checklist.....	15
14. Conclusion.....	16
Appendix A. Detailed QA Test Cases	17
Appendix B. Jira and GitHub Evidence Screenshots	18
B.1 Sprint 1 Evidence	18
B.2 Sprint 2 Evidence	19
B.3 Sprint 3 Evidence	20
B.4 Sprint 4 Evidence	21
B.5 Sprint 5 Final QA Evidence	22

B.6 Jira-GitHub Integration Evidence	23
B.7 Velocity Report.....	24
B.8 GitHub Repository Evidence.....	25

1. Executive Summary

YallaMarket is a full-stack bilingual digital marketplace developed as a Software Engineering course project. The system provides a storefront for digital products such as game cards, subscriptions, balances, and related digital services. It supports Arabic and English interfaces, RTL/LTR layout switching, product browsing, cart management, checkout/order submission, authentication, and admin management features.

The final implementation follows a layered architecture using React, Vite, Tailwind CSS, React Router, and Context API on the frontend; Express.js REST APIs and JWT authentication on the backend; and SQLite for server-side persistence. The development lifecycle was managed through Jira using Agile/Scrum practices, while GitHub was used for collaboration, branching, commits, and pull request evidence.

This report consolidates the engineering process: requirements, system design, implementation, QA testing, bug tracking, Agile evidence, version control evidence, and final submission readiness.

2. Project Overview and Problem Definition

2.1 Problem Definition

Digital marketplace users need a clear, responsive, and localized shopping experience that allows them to browse digital products, manage a cart, submit orders, and interact with a bilingual interface. YallaMarket addresses this need by providing a structured web-based marketplace with customer-facing features and administrative management support.

2.2 Product Scope

- Bilingual storefront interface with Arabic RTL and English LTR support.
- Homepage with category entry points and product navigation.
- Category pages with product listing and search/filter behavior.
- Product details pages showing product information, price, stock status, and related products.
- Shopping cart operations: add, update quantity, remove items, subtotal, delivery fee, total.
- Cart persistence using browser localStorage.
- Checkout flow with customer information, delivery details, payment method selection, and order summary.
- Backend API support for authentication, catalog data, order submission, and admin workflows.
- SQLite database persistence for users, categories, products, orders, and order items.
- Agile project tracking through Jira and collaborative source control through GitHub.

3. Stakeholder Analysis

Stakeholder	Description	Main Interest / Access
Guest	Unauthenticated visitor browsing the storefront.	Browse categories, products, cart, and checkout UI.
Registered User	Customer with an account.	Login, use cart, submit orders, and track personal actions.
Admin	Store manager/operator.	Manage products, categories, orders, and dashboard information.
Development Team	Students building and documenting the system.	Implementation, testing, Jira tracking, GitHub collaboration, and final delivery.
Instructor / Evaluator	Software Engineering course reviewer.	Assess requirements, design, implementation, QA, Agile process, and proof of work.

4. Requirements Summary

4.1 Functional Requirements Summary

ID	Requirement	Implementation Area	Final Status
FR-01	Browse categories	Homepage, category API/service	Implemented

FR-02	View category products	CategoryPage, product API/service	Implemented
FR-03	Search/filter products	Category search and API filtering	Implemented
FR-04	View product details	ProductPage and related products	Implemented
FR-05	Add to cart	ProductCard/ProductPage + CartContext	Implemented
FR-06	Update quantity	CartPage + CartContext	Implemented
FR-07	Remove cart item	CartPage + CartContext	Implemented
FR-08	Persist cart	localStorage under cart state	Implemented
FR-09	View cart summary	OrderSummary/cart totals	Implemented
FR-10	Checkout form	CheckoutPage + order service	Implemented
FR-11	Language switch	LanguageContext + document dir/lang	Implemented
FR-12	Display feedback	Toast/errors/fallback UI	Implemented
FR-13	Register account	Auth API + Register UI	Implemented
FR-14	Login	Auth API + Login UI + JWT	Implemented
FR-15	Logout	Client-side session clearing	Implemented / limited
FR-16	Protected routes	Frontend route guard + backend middleware	Implemented
FR-17	Submit order	POST /api/orders + SQLite persistence	Implemented
FR-18	Admin management	Admin dashboard, products, categories, orders	Implemented

4.2 Non-Functional Requirements Summary

Category	Requirement	Verification Approach
Performance	Primary pages should load within an acceptable time for local/demo use.	Build verification and manual browser testing.
Responsiveness	Support mobile, tablet, and desktop layouts.	Manual UI review using responsive layouts.
Usability	User can move from browsing to checkout through a clear flow.	Manual end-to-end checkout test.
Localization	Arabic RTL and English LTR should be supported.	Language switch and direction test cases.
Reliability	Handle empty states, invalid routes, and API errors gracefully.	Error handling and route fallback tests.
Maintainability	Organized frontend/backend structure with reusable services/components.	Repository structure review.
Security	Use password hashing and JWT authentication.	Auth API and middleware review.
Quality Assurance	Manual test cases, results, and Jira bug tracking evidence.	QA report and Sprint 5 evidence.

5. System Architecture and Technology Stack

YallaMarket follows a three-layer architecture. The presentation layer renders the user interface and manages client-side state. The application/API layer exposes REST endpoints and applies business logic, authentication, validation, and protected route handling. The data layer persists users, categories, products, orders, and order items using SQLite.

Layer	Technology	Responsibility
Presentation Layer	React + Vite + Tailwind CSS + React Router	UI, routing, RTL/LTR, product browsing, cart, checkout, admin screens.
Application/API Layer	Express.js + controllers + middleware	Business logic, REST endpoints, JWT authentication, validation, admin protection.
Data Layer	SQLite	Persistent storage for users, categories, products, orders, and order items.
Client Storage	localStorage	Cart persistence and selected session/auth data where appropriate.
Collaboration & Process	Jira + GitHub	Sprint planning, bug tracking, commits, branching, pull requests, and evidence.

W

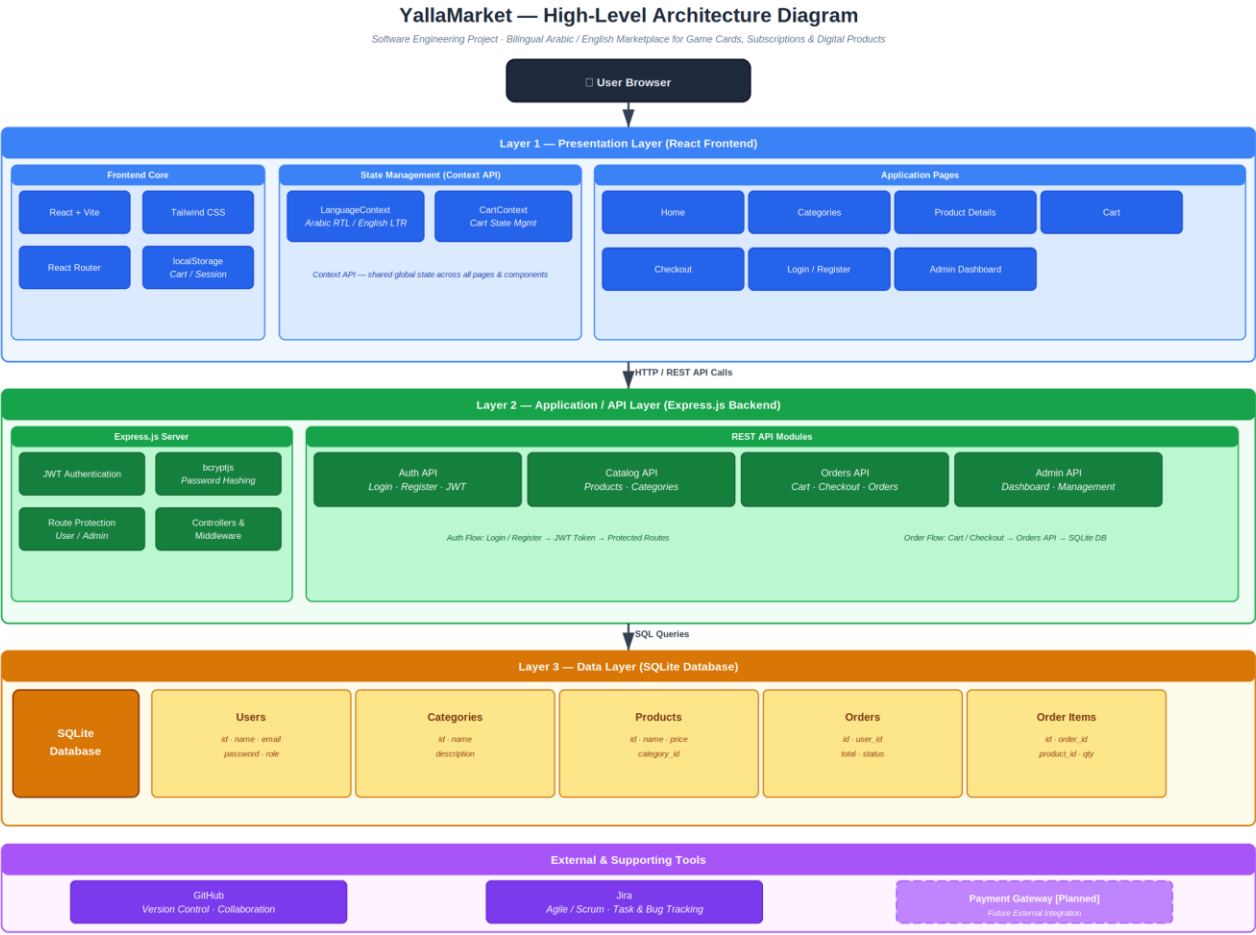


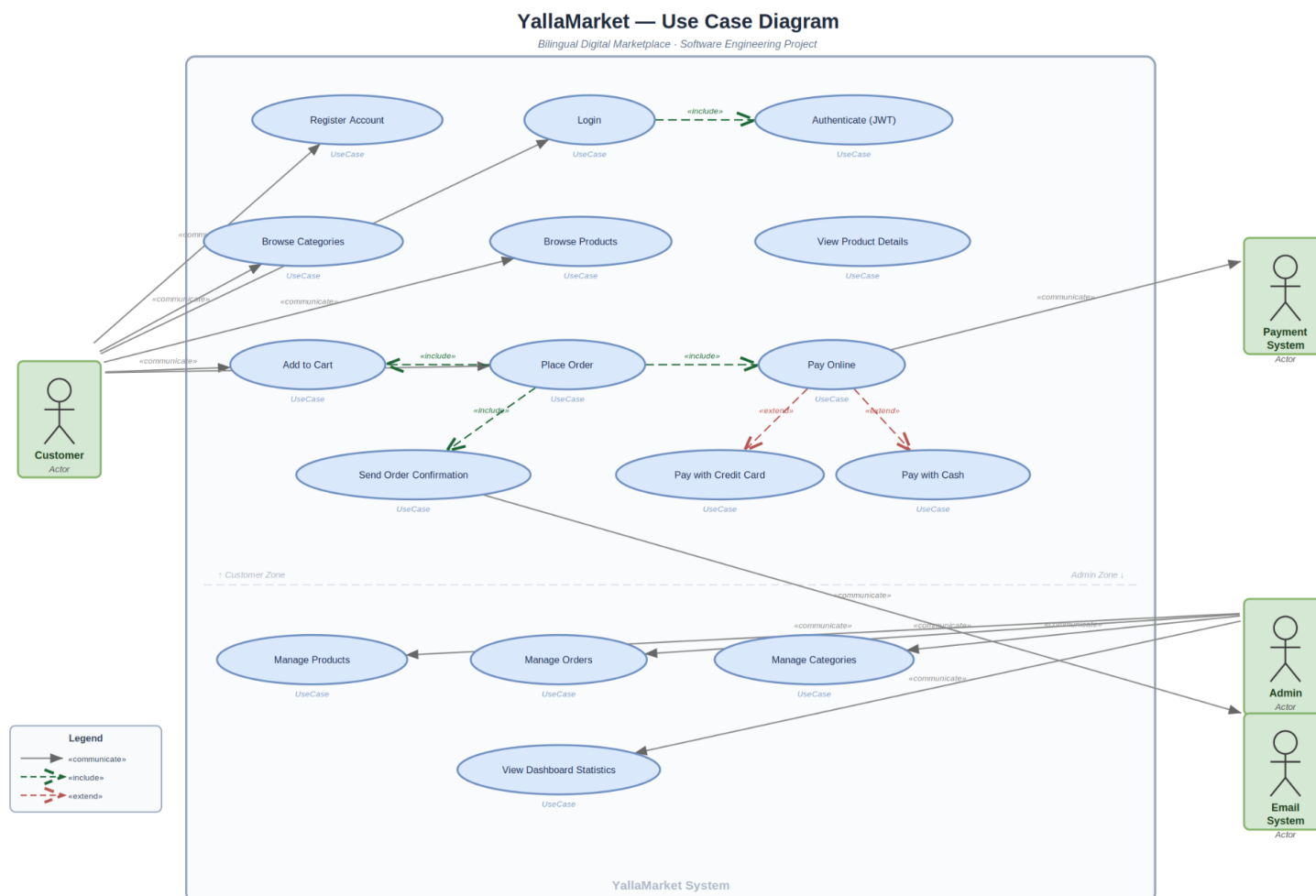
Figure 1. YallaMarket high-level three-layer architecture.

6. System Design Document (SDD)

This section presents the main UML and architecture artifacts used to model YallaMarket. The design diagrams describe users, system behavior, class-level structure, database relationships, and important sequence flows.

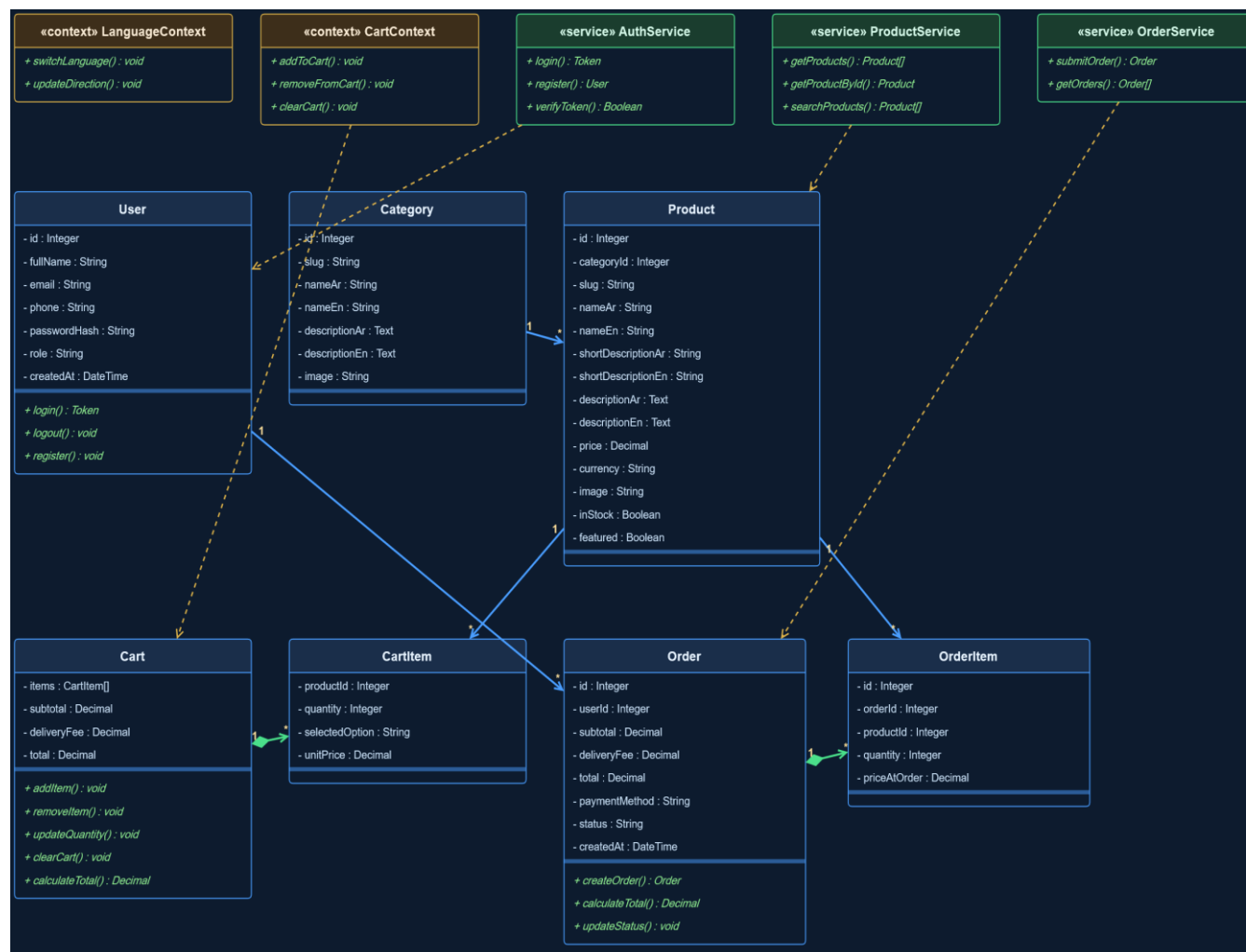
6.1 Use Case Diagram

The use case diagram models the interaction between customers, admins, and external systems. It covers browsing, authentication, cart operations, order placement, payment method selection, and administrative management.



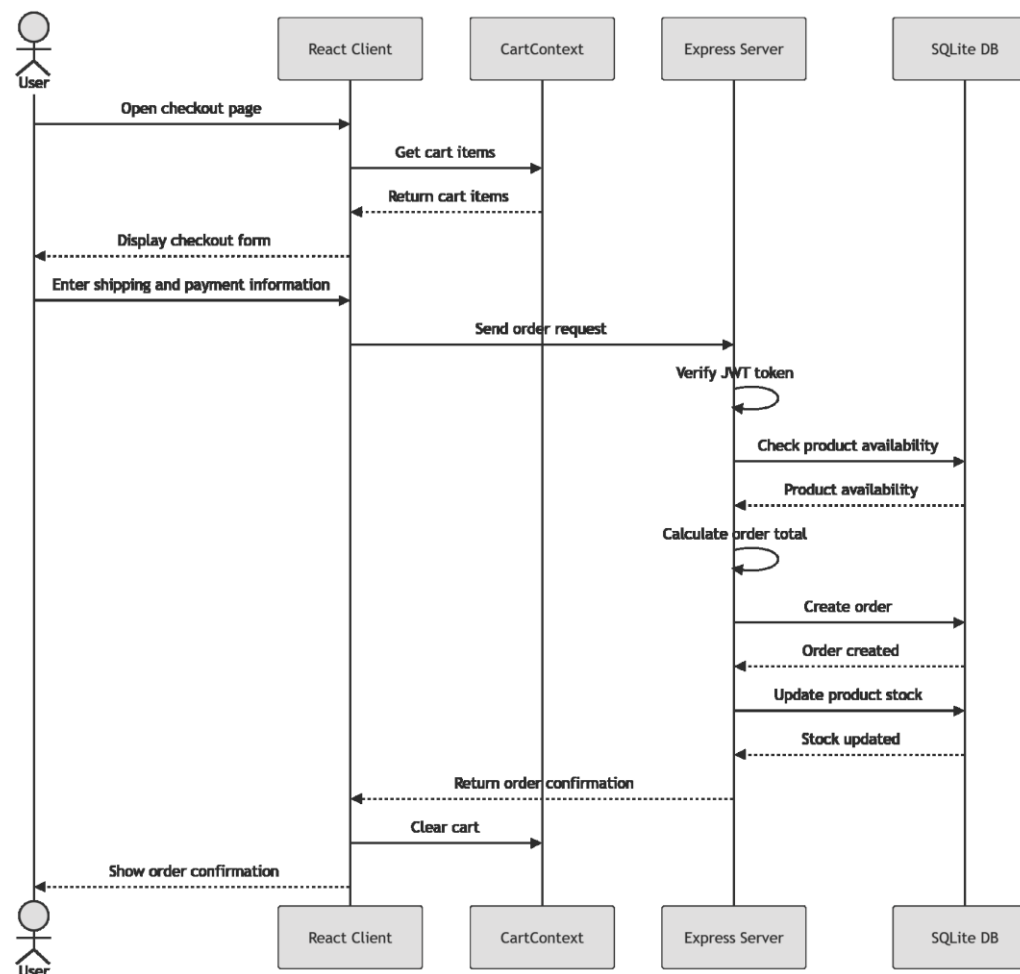
6.2 Class Diagram

The class diagram represents the main domain concepts and supporting client services/contexts, including User, Category, Product, Cart, CartItem, Order, OrderItem, LanguageContext, CartContext, and service modules.

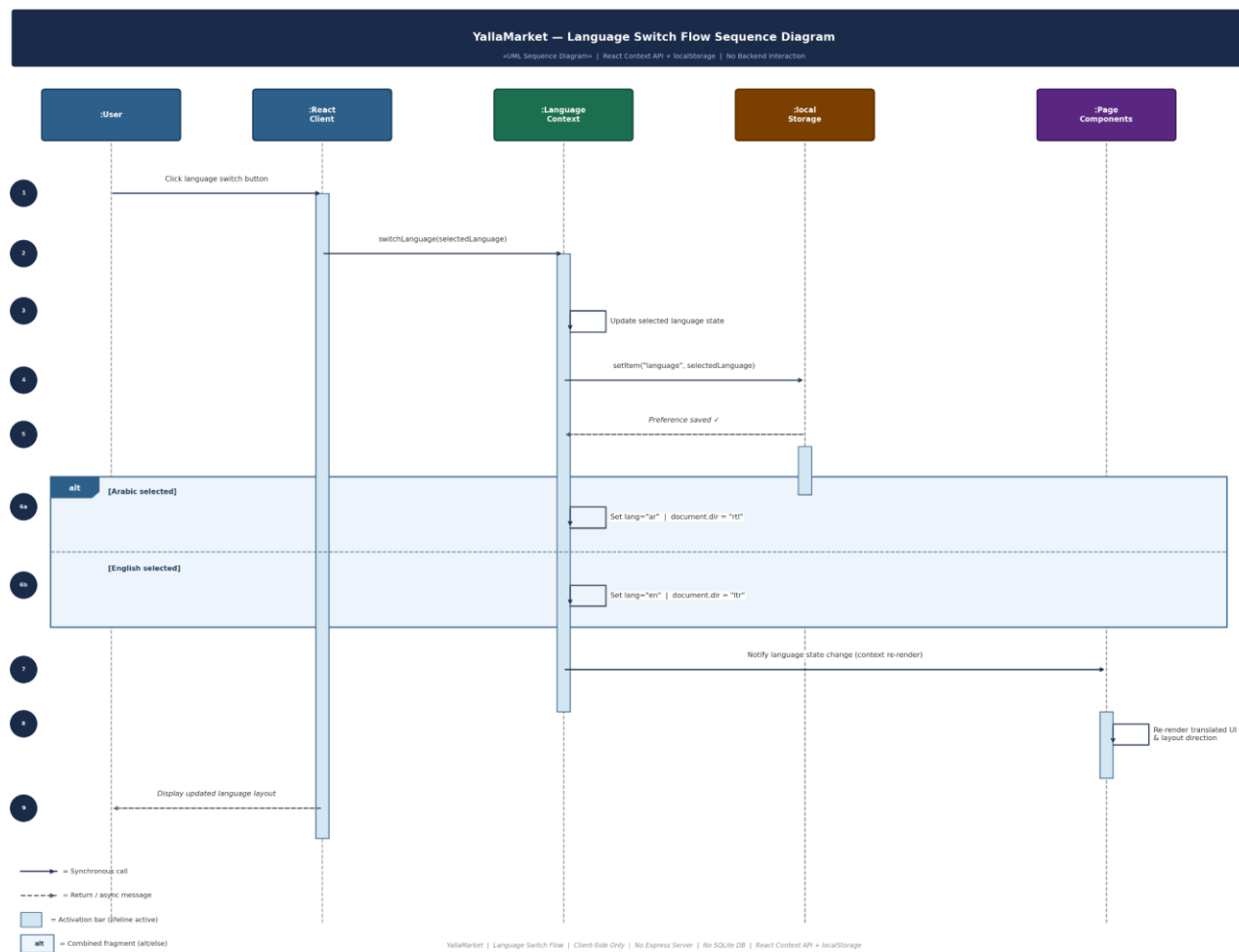


6.3 Sequence Diagrams

Three sequence diagrams document important runtime flows: checkout order submission, login/logout, and language switching. These diagrams demonstrate how frontend state, backend APIs, localStorage, JWT, and SQLite interact during key system operations.



6.3.1 Language Switch Flow



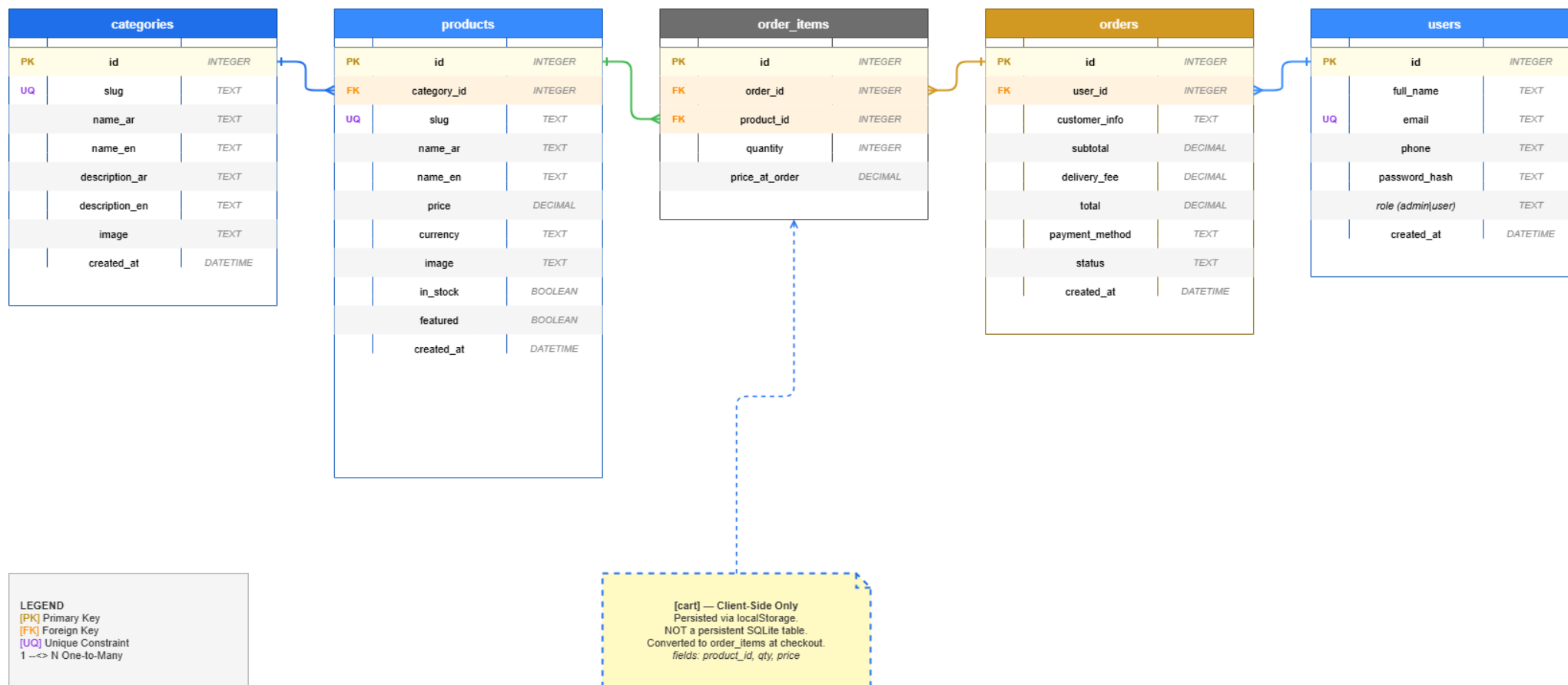
7. Database Design

YallaMarket uses SQLite as a lightweight relational database. The main persistent entities are categories, products, users, orders, and order_items. The cart is intentionally client-side only and is persisted in localStorage until the checkout flow converts cart items into order_items.

7.1 Entity Relationship Diagram

YallaMarket — Entity Relationship Diagram

Database: SQLite · Backend: Express.js · Auth: JWT · Admin via users.role · Cart: localStorage



7.2 Data Model Summary

Entity	Purpose	Key Fields / Relationships
categories	Stores product categories.	id, slug, name_ar/name_en, descriptions, image. One category has many products.
products	Stores purchasable digital products.	id, category_id, names, descriptions, price, image, stock/availability fields.
users	Stores customer/admin accounts.	id, full_name, email, phone, password_hash, role, created_at.
orders	Stores checkout orders.	id, user_id, customer_info, totals, payment_method, status, created_at.
order_items	Stores items belonging to an order.	id, order_id, product_id, quantity, price_at_order.
client cart	Temporary client-side cart before checkout.	Stored in localStorage; converted to order_items during order submission.

8. Implementation Overview

8.1 Frontend Implementation

- React + Vite project structure under client/src.
- Customer pages: Home, CategoryPage, ProductPage, CartPage, CheckoutPage, LoginPage, RegisterPage, NotFoundPage.
- Admin pages: AdminDashboard, AdminProducts, AdminCategories, AdminOrders.
- Reusable components include Layout, Navbar, Footer, ProductCard, CategoryCard, CartItem, QuantitySelector, OrderSummary, Toast, AuthForm, AdminRoute, and Spinner.
- Context providers handle language, cart, and authentication state.
- LanguageContext updates document language and direction and persists user language preference.
- CartContext manages add/update/remove/clear behavior and persists the cart in localStorage.
- AuthContext stores the JWT and authenticated user data.

8.2 Backend Implementation

- Express.js backend under server/src.
- Main app/server files handle middleware, routing, and application startup.
- Auth routes support register, login, and logout behavior.
- Product and category routes support public catalog access and admin CRUD operations.
- Order routes support authenticated order creation, user order retrieval, and admin order management.
- JWT middleware protects authenticated routes; admin middleware protects role-based actions.
- Passwords are hashed using bcrypt/bcryptjs and tokens are signed/verified using jsonwebtoken.
- SQLite schema and data are initialized through backend database configuration.

8.3 API Summary

Method	Endpoint	Purpose	Auth	Status
GET	/api/health	API health check	No	Implemented
POST	/api/auth/register	Create user account	No	Implemented
POST	/api/auth/login	Authenticate user and return JWT	No	Implemented
POST	/api/auth/logout	Logout / clear client session	Token optional	Implemented / limited
GET	/api/categories	List categories	No	Implemented
GET	/api/categories/slug/:slug	Get category by slug	No	Implemented
POST	/api/categories	Create category	Admin	Implemented
PUT	/api/categories/:id	Update category	Admin	Implemented
DELETE	/api/categories/:id	Delete category	Admin	Implemented
GET	/api/products	List/search/filter products	No	Implemented
GET	/api/products/:id	Product details	No	Implemented
GET	/api/products/related/:id	Related products	No	Implemented
POST	/api/products	Create product	Admin	Implemented
PUT	/api/products/:id	Update product	Admin	Implemented
DELETE	/api/products/:id	Delete product	Admin	Implemented
POST	/api/orders	Submit order	User	Implemented
GET	/api/orders/my	Current user orders	User	Implemented
GET	/api/orders	Admin order list	Admin	Implemented
GET	/api/orders/:id	Order details	Owner/Admin	Implemented
PUT	/api/orders/:id/status	Update order status	Admin	Implemented
GET	/api/admin/dashboard/stats	Dashboard statistics	Admin	Implemented

9. Quality Assurance

Quality assurance focused on manual functional testing, integration testing, UI validation, authentication validation, cart persistence, checkout/order flow, admin functionality, localization, and route/error handling. Testing was documented as part of the final QA sprint and connected with Jira bug tracking evidence.

9.1 Test Plan Summary

Area	QA Approach
Scope	Storefront, categories, products, cart, checkout, auth, admin, localization, routing, and error handling.
Environment	Local frontend/backend environment, modern browser, SQLite database with seeded/demo data.
Testing Type	Manual functional testing, integration testing, UI/UX verification, regression checks.
Entry Criteria	Frontend and backend run successfully; database available; main pages accessible.
Exit Criteria	Core test cases pass; discovered bugs are resolved/verified; Jira evidence captured.

9.2 Test Execution Summary

Metric	Value	Notes	Result
Manual Test Cases	20	Covers storefront, cart, auth, checkout, admin, localization, and error handling.	Completed
Passed	20	All final manual test cases passed after final QA fixes.	Pass
Failed	0	No open blocking failures at final QA sign-off.	Pass
Jira Bugs Tracked	4	SCRUM-127 to SCRUM-130.	Resolved
Final QA Tasks	3	SCRUM-131 to SCRUM-133.	Done

9.3 Bug Summary

Jira Key	Bug Summary	Severity	Resolution	Status
SCRUM-127	Cart total did not update immediately after quantity changes.	Medium	Cart recalculation behavior verified and corrected.	Done
SCRUM-128	Checkout validation messages were unclear for incomplete forms.	Medium	Validation feedback improved and verified.	Done
SCRUM-129	Language direction was inconsistent after switching languages.	Medium	RTL/LTR behavior verified and synchronized.	Done
SCRUM-130	Invalid product routes needed improved fallback handling.	Medium	NotFound/fallback behavior improved and verified.	Done

9.4 Requirements Traceability Matrix

Requirement Area	Implementation Area	Related Test Cases	Result
Browse categories	HomePage + category API	TC-01, TC-02	Pass
Search/filter products	CategoryPage + product service	TC-03, TC-04	Pass
Product details	ProductPage + related products	TC-05, TC-06	Pass
Cart operations	CartContext + CartPage	TC-07 to TC-10	Pass
Authentication	Auth API + AuthContext	TC-11 to TC-13	Pass
Checkout/orders	CheckoutPage + Order API	TC-14 to TC-16	Pass
Admin features	Admin dashboard/products/orders	TC-17, TC-18	Pass
Localization	LanguageContext + RTL/LTR	TC-19	Pass
Error handling	NotFound/fallback UI	TC-20	Pass

10. Agile Project Management Evidence

The project was managed using Agile/Scrum in Jira. The backlog included requirements, design activities, frontend tasks, backend tasks, integration tasks, deployment/testing tasks, and a final QA sprint. Sprint 5 was used for final quality assurance, bug tracking, documentation, presentation preparation, and delivery evidence.

Sprint	Focus	Representative Work	Status
--------	-------	---------------------	--------

Sprint 1	Project setup and early frontend foundation	React + Vite setup, homepage, navbar, repository setup, SRS and initial diagrams.	Completed
Sprint 2	Frontend stabilization and core storefront	CartContext, cart page, CategoryPage, ProductCard, language support, UX refinements.	Completed
Sprint 3	Backend and database integration	Node/Express setup, SQLite models, Auth API, products/categories/orders REST APIs, sample data.	Completed
Sprint 4	Full-stack integration and admin functionality	API connection, admin dashboard/products/orders, deployment/testing, Postman API validation.	Completed
Sprint 5	Final QA and delivery preparation	Bugs SCRUM-127..130, QA documentation, final presentation/demo flow, Jira/GitHub evidence.	Completed

11. Version Control and Collaboration Evidence

GitHub was used as the team repository for collaborative implementation. Evidence includes commit history, branches, pull requests, merge activity, and Jira integration through issue keys. The repository URL is: <https://github.com/BaraaMasri99/yallaMarket>

Evidence Type	What It Demonstrates	Included Evidence
Repository overview	Project structure and final working codebase.	GitHub repository screenshot.
Commit history	Incremental implementation and maintenance.	Commit count and recent commit activity.
Branches	Team collaboration and task separation.	Branch count evidence.
Pull requests	Integration workflow and Jira issue linking.	Merged PR screenshot with Jira issue references.
Jira integration	Traceability between project management and code changes.	GitHub/Jira link screenshot.

12. Known Limitations and Future Enhancements

- Real external payment gateway integration is not included; the current system supports payment method selection and order recording.
- Logout mainly clears the client-side token/session; server-side token invalidation can be improved in future versions.
- The system uses SQLite for lightweight academic/demo persistence; a production deployment could migrate to a managed relational database.
- Automated test coverage can be added in future work; the current submission provides structured manual QA testing and Jira bug tracking evidence.
- Additional deployment hardening, monitoring, and production secrets management can be added beyond the course scope.

13. Final Submission Checklist

Submission Item	Evidence / File	Status
Final Report PDF	This comprehensive report exported to PDF.	Ready for export
SRS	SRS v3.0 included/summarized in the report.	Ready
System Design Document	Use Case, Class, Sequence diagrams, Architecture, ERD.	Ready
QA Documentation	QA report, test cases, test results, bug summary.	Ready
Agile Evidence	Jira backlog, sprints, Sprint 5 bugs/tasks, velocity report.	Ready
GitHub Evidence	Repository URL, commits, branches, pull requests, Jira links.	Ready

Source Code ZIP	Final working project archive.	Prepare before upload
Presentation PPTX	Final presentation slides.	Prepare/polish
Video Recording	All members participate in presentation and proof-of-work demo.	Record after rehearsal

14. Conclusion

YallaMarket demonstrates a complete Software Engineering lifecycle for a full-stack digital marketplace: requirements definition, stakeholder analysis, system design, implementation, testing, Agile tracking, and final delivery preparation. The project includes a working bilingual storefront, authentication, cart persistence, checkout/order flow, admin management, SQLite persistence, Jira evidence, GitHub collaboration evidence, and QA documentation.

The final focus was not to redesign the product, but to stabilize the existing implementation, document quality assurance, resolve final QA issues, and prepare a professional submission package. Based on the completed implementation and documentation artifacts, the project is ready for final presentation and course submission after final packaging and video recording.

Appendix A. Detailed QA Test Cases

TC ID	Feature	Scenario	Expected Result	Actual Result	Status
TC-01	Home / Categories	Open homepage	Categories are displayed correctly.	Categories loaded successfully from API.	Pass
TC-02	Categories	Open a category page	Related products are displayed.	Products displayed successfully.	Pass
TC-03	Search	Search using an Arabic keyword	Matching Arabic products appear.	Arabic filtering works correctly.	Pass
TC-04	Search	Search using an English keyword	Matching English products appear.	English filtering works correctly.	Pass
TC-05	Product Details	Open product details page	Product name, price, description, and stock are visible.	Product details loaded successfully.	Pass
TC-06	Related Products	View related products section	Related products are displayed.	Related products displayed successfully.	Pass
TC-07	Cart	Add available product to cart	Cart count and item list update.	Product added successfully.	Pass
TC-08	Cart	Update item quantity	Quantity and totals update correctly.	Totals update after quantity changes.	Pass
TC-09	Cart	Remove item from cart	Item is removed and totals recalculate.	Item removed successfully.	Pass
TC-10	Cart Persistence	Refresh browser after adding cart items	Cart remains saved.	Cart persisted using localStorage.	Pass
TC-11	Authentication	Register new user	Account is created successfully.	Registration completed successfully.	Pass
TC-12	Authentication	Login with valid credentials	User is authenticated and token stored.	JWT authentication works correctly.	Pass
TC-13	Authentication	Login with invalid credentials	Clear error message is displayed.	Invalid login feedback displayed.	Pass
TC-14	Checkout	Open checkout with empty cart	User sees empty cart/checkout restriction.	Empty cart behavior handled.	Pass
TC-15	Checkout	Submit checkout while logged out	System requires authentication.	Login/authentication requirement displayed.	Pass
TC-16	Checkout	Submit valid order while logged in	Order is created and saved.	Order submitted successfully.	Pass
TC-17	Admin	Admin dashboard access	Dashboard statistics are visible to admin.	Admin dashboard loaded correctly.	Pass
TC-18	Admin Orders	Update order status	Order status updates and persists.	Order status updated successfully.	Pass
TC-19	Localization	Switch Arabic/English	Language and direction update correctly.	RTL/LTR switch verified.	Pass
TC-20	Routing/Error Handling	Open invalid product/category URL	Fallback or NotFound message appears.	Fallback handling verified.	Pass

Appendix B. Jira and GitHub Evidence Screenshots

B.1 Sprint 1 Evidence

Work	Assignee	Reporter	Priority	Status	Resolution	Created	
☒ SCRUM-100 Initialize React + Vite project and configure T...	 BaraaMasri99	 BaraaMasri99	 Highest	 DONE	Done	Mar 30, 2026, 2:26 PM	
 SCRUM-84 Build Home page with hero banner and categ...	 BaraaMasri99	 BaraaMasri99	 High	 DONE	Done	Mar 30, 2026, 2:13 PM	
 SCRUM-83 Build Navbar component with search and cart...	 BaraaMasri99	 BaraaMasri99	 High	 DONE	Done	Mar 30, 2026, 2:13 PM	
☒ SCRUM-82 Create GitHub repository and set up branchin...	 Hamed Bizreh	 BaraaMasri99	 Highest	 DONE	Done	Mar 30, 2026, 2:13 PM	
 SCRUM-80 Draw UML Use Case diagram	 Yahya Zyoud	 BaraaMasri99	 Medium	 DONE	Done	Mar 30, 2026, 2:12 PM	
 SCRUM-79 Design ERD (Entity Relationship Diagram)	 Hamed Bizreh	 BaraaMasri99	 High	 DONE	Done	Mar 30, 2026, 2:12 PM	
 SCRUM-78 Write SRS document 	 BaraaMasri99	 BaraaMasri99	 High	 DONE	Done	Mar 30, 2026, 2:12 PM	

Figure 8. Sprint 1 completed work items.

B.2 Sprint 2 Evidence

Work	Assignee	Reporter	Priority	Status	Resolution	Created	
 SCRUM-111 Refine frontend UX and improve state manage...	 BaraaMasri99	 BaraaMasri99	 Medium		Done	Apr 15, 2026, 6:03 PM	
 SCRUM-109 Arabic and English lang	 BaraaMasri99	 BaraaMasri99	 Medium		Done	Apr 15, 2026, 5:33 PM	
 SCRUM-98 Draw UML Class and Architecture diagrams	 Hamed Bizreh	 BaraaMasri99	 Low		Done	Mar 30, 2026, 2:18 PM	
 SCRUM-88 Build CartContext	 Hamed Bizreh	 BaraaMasri99	 High		Done	Mar 30, 2026, 2:13 PM	
 SCRUM-87 Build Cart page with order summary	 BaraaMasri99	 BaraaMasri99	 High		Done	Mar 30, 2026, 2:13 PM	
 SCRUM-86 Build CategoryPage with product grid	 Yahya Zyoud	 BaraaMasri99	 High		Done	Mar 30, 2026, 2:13 PM	
 SCRUM-85 Build reusable ProductCard component	 BaraaMasri99	 BaraaMasri99	 High		Done	Mar 30, 2026, 2:13 PM	
 SCRUM-78 Write SRS document	 BaraaMasri99	 BaraaMasri99	 High		Done	Mar 30, 2026, 2:12 PM	

Figure 9. Sprint 2 completed work items.

B.3 Sprint 3 Evidence

Work	Assignee	Reporter	Priority	Status	Resolution	Created	
✓ SCRUM-94 Insert sample Arabic and english data into SQ...	HB Hamed Bizreh	B BaraaMasri99	= Medium	DONE ✓	Done	Mar 30, 2026, 2:18 PM	
🔖 SCRUM-93 Build Products, Categories, and Orders REST ...	YZ Yahya Zyoud	B BaraaMasri99	^ High	DONE ✓	Done	Mar 30, 2026, 2:18 PM	
🔖 SCRUM-92 Build Auth API (register, login, logout,JWT)	B BaraaMasri99	B BaraaMasri99	^ High	DONE ✓	Done	Mar 30, 2026, 2:14 PM	
🔖 SCRUM-91 Create all SQLite database models	YZ Yahya Zyoud	B BaraaMasri99	^ High	DONE ✓	Done	Mar 30, 2026, 2:14 PM	
✓ SCRUM-90 Initialize Node.js + Express with SQLite	B BaraaMasri99	B BaraaMasri99	⤴ Highest	DONE ✓	Done	Mar 30, 2026, 2:14 PM	
🔖 SCRUM-89 Build Login and Register page UI	B BaraaMasri99	B BaraaMasri99	= Medium	DONE ✓	Done	Mar 30, 2026, 2:13 PM	

Figure 10. Sprint 3 backend and integration items.

B.4 Sprint 4 Evidence

Work	Assignee	Reporter	Priority	Status	Resolution	Created	
☒ SCRUM-116 Test the api using postman	 Hamed Bizreh	 BaraaMasri99	 Medium		Done	Apr 22, 2026, 11:36 PM	
☒ SCRUM-99 Deploy frontend to Vercel and backend to Ren...	 Hamed Bizreh	 BaraaMasri99	 Low		Done	Mar 30, 2026, 2:19 PM	
 SCRUM-97 Connect frontend to backend API	 BaraaMasri99	 BaraaMasri99	 High		Done	Mar 30, 2026, 2:18 PM	
 SCRUM-96 Build Admin Manage Products and Manage Or...	 Yahya Zyoud	 BaraaMasri99	 Medium		Done	Mar 30, 2026, 2:18 PM	
 SCRUM-95 Build an admin dashboard with stats overview	 Yahya Zyoud	 BaraaMasri99	 Medium		Done	Mar 30, 2026, 2:18 PM	

Figure 11. Sprint 4 integration, admin, deployment/testing items.

B.5 Sprint 5 Final QA Evidence

Work	Assignee	Reporter	Priority	Status	Resolution	Created	
✓ SCRUM-133 Capture Jira and GitHub evidence for final re...	 Hamed Bizreh	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:28 PM	
✓ SCRUM-132 Prepare final presentation video and demo fl...	 BaraaMasri99	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:27 PM	
✓ SCRUM-131 Finalize QA documentation and manual test e...	 BaraaMasri99	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:27 PM	
✖ SCRUM-130 Invalid product routes needed improved fallb...	 Yahya Zyoud	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:25 PM	
✖ SCRUM-129 Language direction was inconsistent after sw...	 Yahya Zyoud	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:24 PM	
✖ SCRUM-128 Checkout validation messages were unclear f...	 Hamed Bizreh	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:24 PM	
✖ SCRUM-127 Cart total did not update immediately after qu...	 Hamed Bizreh	 BaraaMasri99	⚖ Medium	DONE ▾	Done	May 10, 2026, 10:24 PM	

Figure 12. Sprint 5 final QA bugs and tasks marked Done.

B.6 Jira-GitHub Integration Evidence

Summary	Status ↑	Owner	Reviewers	Updated
 SCRUM-92 register & login UI  Repository: BaraaMasri99/yallaMarket 	MERGED		No reviewers	19 days ago
 SCRUM-97 replace mock data with API calls  Repository: BaraaMasri99/yallaMarket 	MERGED		No reviewers	18 days ago

Figure 13. Merged pull requests linked to Jira issue keys.

B.7 Velocity Report

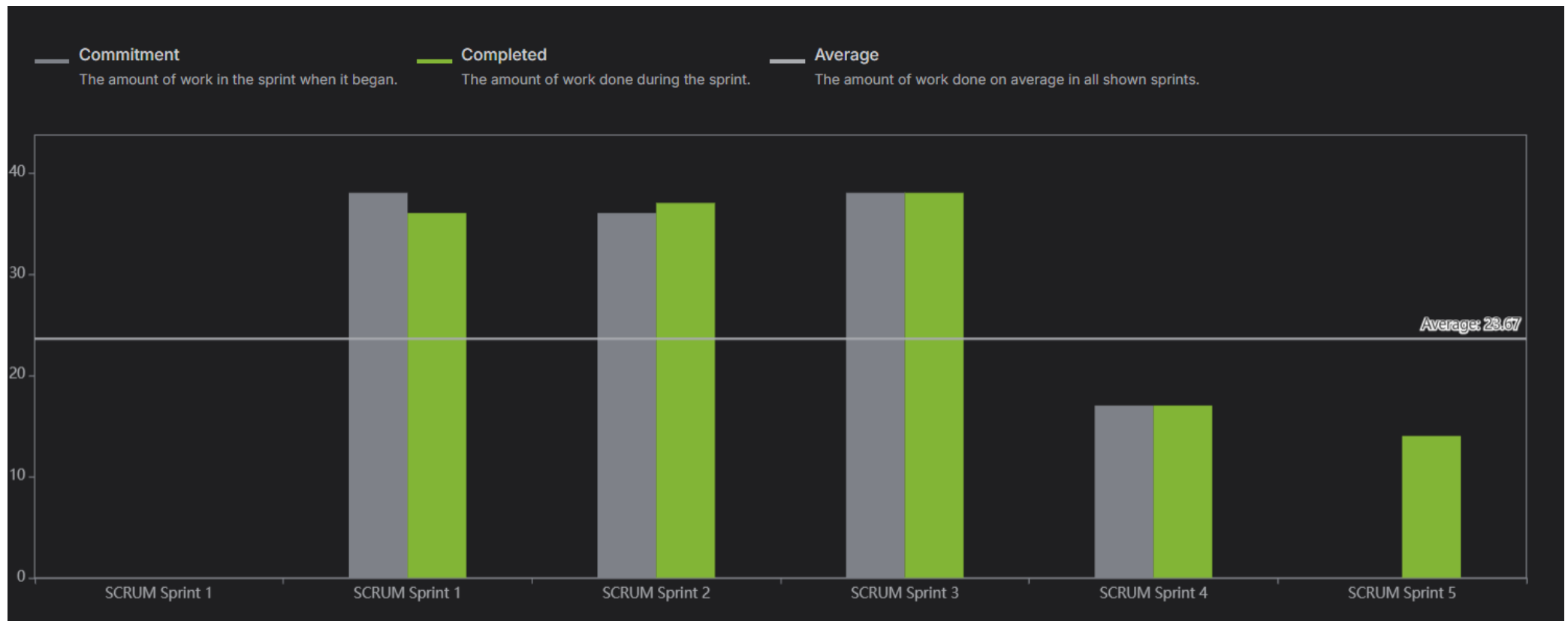


Figure 14. Jira velocity report showing sprint progress.

B.8 GitHub Repository Evidence

The screenshot shows the GitHub repository page for 'yallaMarket', which is a private repository. The repository is currently on the 'main' branch, with 11 other branches and 0 tags. The commit history shows a recent commit by BaraaMasri99 with the message 'add some pictures for the products' (commit da9fd8b, 2 days ago). The repository has 46 commits in total. The file list includes a 'client' folder, a 'docs' folder, a 'server' folder, and several files: '.gitignore', 'README.md', 'SECURITY.md', and 'script.sh'. The commit history for each file is also displayed.

File/Folder	Commit Message	Commit Time
client	add some pictures for the products	2 days ago
docs	remove useless file	2 days ago
server	add some pictures for the products	2 days ago
.gitignore	Initialize Node.js + Express server. Configure CORS, dotenv, J...	3 weeks ago
README.md	fixing some issues	last week
SECURITY.md	Create SECURITY.md for security policy	2 days ago
script.sh	Update the Page layout and make a script to run the fronten...	2 weeks ago

Figure 15. GitHub repository overview showing project structure and commit history.